Page Checks — Technical Reference

An engineering-oriented reference for the checks in **Is It Agent-Ready? for SitecoreAI**: the scoring model, every check with its thresholds and credit, and where each one lives in the code. The Page Builder panel ("Is this page agent-ready?") and the dashboard ("Is this site agent-ready?") use **the same engine**, so the same inputs give the same score in both. The panel also scores the Page Builder **draft**, so its score follows unpublished edits (see "Draft and live" below). Dashboard specifics (roll-up, cache, site checks card) are in `PAGE-MATRIX-TECHNICAL.md`.

Source of truth: `src/lib/checks/` — `registry.ts` (the check list), `page/*.ts` and `site/*.ts` (the checks), `scoring.ts` (pool maths), `gate.ts`, `engine.ts` (entry points), `context.ts` (inputs), `draft-diff.ts` (draft vs live comparison), `json-ld.ts` (shared JSON-LD parser), `view-model.ts` (what the UI shows). HTML parsing is in `src/lib/html/`. Plan reference: `docs/IS-IT-AGENT-READY-PLAN.md` sections 5 and 7.

Scoring model

- Every check belongs to one of **four categories**, phrased as questions: **Reach** (Can AI agents get to it?), **Read** (Can they read it?), **Understand** (Do they understand it?), **Trust** (Will they trust and cite it?).
- Every check has a **tier**. **Essential** checks share an **80-point pool**, **Recommended** checks a **20-point pool**, and **Bonus** checks add **up to +5**. The total is capped at 100.
- A check returns a **credit from 0 to 1** (partial credit is normal) and a status (`pass` / `warning` / `fail` / `info`). Its points are `credit × its share of the tier pool`. Shares are equal unless a check sets the optional `weight` multiplier (default 1; none are set today).
- A check can also return **N/A** (`not-applicable`) or **not checked** (`not-checked` : AI pending, PageSpeed not run, fetch failed). Both are **left out of the pools, never failed**. The pool is split across the checks that are scored, so a page is only graded on what applies to it.
- If Essential or Recommended has nothing scored, the other one is scaled up so the base still totals 100. With nothing scored at all, there is no score yet (`null`).
- The score is rounded to a whole number. Exact points are kept per check and per category (`ScoreBreakdown` in `types.ts`).
- Thresholds are shared by the panel and the dashboard (`src/lib/score-thresholds.ts`): **Agent-ready ≥ 80**, **Almost there 60–79**, **Not ready < 60**.

Points. Five Essential page checks exist (P2, P3, P8, P9, P10). In the panel, when all apply, each is worth 16 points. On the dashboard P3 isn't checked (no draft), so the other four are worth 20 each. On the home

page P10 is usually N/A. The Recommended checks share 20 points (about 1.7 each when all twelve apply). Phase 7 adds P5 to Essential.

Scopes. Page checks run per page in the panel and in dashboard scans. Site checks run once per site and get **their own site score**; they are never part of a page score. There are two site checks so far: S2 and S7.

Same inputs, same score. Both views build the check context with `buildPageContext` (`context.ts`) from the same inputs: the page's live HTML (`/api/preview-fetch` → `ParsedHtml`), the AI results (`/api/ai-check`, cached by content hash), and the same Sitecore fields read by the same names (`SCORING_FIELD_NAMES`: title, meta description, `robotsMetaTag`, `__Updated`). The panel reads them with `pageFieldsFromFieldMap`, the dashboard with `pageFieldsFromValues`. `src/lib/checks/parity.test.ts` covers this.

Where the panel and the dashboard differ, by design:

- **The draft.** The panel's body checks and AI checks read the Page Builder draft. It includes unpublished edits and content rendered by JavaScript (e.g. BrandX's Related Articles cards), so the panel's score can differ from the dashboard's even for a page with no edits. Agreed on 2026-11-01 (plan section 7, Phase 4).
- **P3** needs the draft, so it's only scored in the panel. The dashboard shows it as "Page Builder only" (not checked, left out of the pools).
- **PageSpeed:** only the panel can run it, so after you run it there, the panel includes P7 and the dashboard doesn't.

Draft and live (plan section 7)

Data	Source	Read by
Draft: `client.getPageHTML()`, sanitised and parsed in the browser	`usePageHtml` (`src/hooks/usePageHtml.ts`), panel only	Body checks: P3, P4, P8, P12, P13, P14, P15, P19, P20. AI body: P16, P21 and the meta generator
Live: `/api/preview-fetch` (server HTML, with the editing secret)	`useScan` (panel), `useSitePages` (dashboard)	Head checks: P2, P9 (fallback), P10, P11, P17, P18, the gate, S7. P3's comparison
Fields: authoring GraphQL	`usePageFields`	P9, P18, the gate

- **Context.** `PageInput.draft` is optional: the dashboard leaves it out. `buildPageContext` sets `body` to the draft's `ParsedHtml` when there is one, otherwise the live one, and `bodySource` to `"draft"` or `"live"`. Body checks read `ctx.body`; head checks read `ctx.html`.

- **P10 stays fully live.** Its JSON-LD (only in the `<head>`, which the draft lacks) and its Q&A pairs come from the same render.
- **Source chip.** A check declares `sources` (body checks declare `"draft"`). `resolveSources` (`engine.ts`) turns `"draft"` into the body's actual source and adds it to AI checks, and every `CheckOutcome` carries the result as `sources`. The panel shows it as a chip next to each check's title (Draft / Live / Fields / AI / PageSpeed; "Draft vs Live" for P3), with the full wording when the row is open.
- **Refresh.** `usePageHtml` reads the draft 1.5 s after a page change (so the canvas can load the new page) and 1.5 s after the last `pages.content.fieldsUpdated` (`usePageFields.contentVersion`), and at once on **Re-scan page**. Only the latest read may set state. Each read takes about 30–50 ms. Console: `[draft] getPageHTML: N chars …; removed {…}`.
- **Retry while the canvas loads.** Right after a page opens, Page Builder answers `getPageHTML` with `CoreError: [client SDK] [CANVAS BUG] Service implementation is not provided` until its canvas has loaded (seen on production on 2026-11-02; a Re-scan a moment later worked). So while a page has no draft yet, a failed read is retried after 2, 4 and 8 s (`readWithRetry`, about 15 s in all). The status stays `loading` meanwhile, so rows show "Scanning…" and the AI checks wait for the draft. Console: `[draft] getPageHTML not ready (attempt N); trying again in N s`. A newer read (page change, field update, Re-scan) stops the retries.
- **Fallback.** If every attempt fails, the status is `unavailable` and the body checks read live, as before: chips say Live, and P3 says "Draft not available … Re-scan to try again" (`draftUnavailable` in the context; the dashboard's "Page Builder only" is unchanged). A failed refresh of a page that already has a draft isn't retried and keeps that draft, so the score doesn't jump mid-edit.
- **Unpublished pages.** If preview-fetch fails (no published page yet), the head checks are not checked and the body checks still score from the draft.
- **AI checks.** They judge the body text (`useScan.bodyText`: the draft's `mainText`, or live's without a draft). They run once per page load and per Re-scan, on the first ready body, never on every edit. When the text has changed since, the AI rows say "Based on an earlier version of the text" with a **Re-run** link (`useScan.aiStale`). Results are cached by content hash as before.
- **"Draft differs from live" banner** (`src/components/panel/DraftBanner.tsx`: "Scored from your draft. It differs from the live page …"). `draftDiffersFromLive` (`draft-diff.ts`) compares the draft's and live's `mainText` line by line. It shows when a live line is missing from the draft, or a draft line is an edit of a live one (word overlap ≥ 0.5). Lines only in the draft, with no live counterpart, don't count: they're usually client-rendered, and would otherwise show the banner on every page with a client-fetched listing.

The sanitiser (`src/lib/html/sanitise-draft.ts`)

Every capture goes through `sanitiseDraftHtml` before it's parsed, so no check sees raw editor output. It removes (SPIKE-FINDINGS §3, item 5, and §3.1 for the captures it's tested on):

- **Page Builder help text** (`.ee-help-text-block` , seen in all three BrandX captures):

- "Placeholder: custom-headless-hero"-style labels around each placeholder go.

- A `Fallback: …` **hint** next to the field's value goes. In the FAQ h1, `<span>Frequently Asked Questions</span><div class="undefined ee-help-text-block">Fallback:<!— —> <span>Frequently Asked Questions</span></div>` becomes one "Frequently Asked Questions".

- When the field is empty, the published page shows the fallback, so the hint is replaced by its value without the label, and the heading keeps its text.

- `[No text in field]` placeholders. Empty headings and paragraphs are then dropped by the extractor.
- **Empty fields' placeholder elements** (class `scEmpty*`), removed whole. The St. Patrick's Day hero has an empty image field, which the editor shows as a grey picture with `alt=""` ; left in, P15 would count an image without alt text that the published page doesn't have. The class is read from the raw attributes, because that `<img>` 's `src` is an inline SVG containing `class="st3"` , which node-html-parser takes as the element's class.
- **Empty-state messages** (`isEmptyStateMessage` : "No articles found", "No results", "Nothing to show", "Loading…"). They're returned as `emptyStates` for P3's note.
- `contenteditable` and the editor-only classes `placeholder` and `editing-mode` . `data-component` and `data-icon` are the site's own and stay.

`{{…}}` template tokens (the footer's `© {{year}}`) are kept: the published server HTML has the value (`© 2026`), and the comparison matches a token to any one word.

The gate — P1 `reach.indexable`

Location: `gate.ts`

Runs before any check. The page is **blocked** when:

- the Sitecore **Robots Meta Tag** field (`robotsMetaTag`) contains `noindex` , **or**
- the rendered `<meta name="robots">` or `<meta name="googlebot">` contains `noindex` , `nosnippet` or `max-snippet:0` .

A blocked page shows the gate message instead of a score (`PageReport.score` is `null` , no outcomes). The dashboard reads the same field (it was added to its page query), shows the row as "Blocked (noindex)", and leaves it out of the averages.

Page checks

Body checks read the **draft** in the panel and the **live HTML** elsewhere; head checks always read the live HTML (table above). "Not yet scanned" (not checked) means the HTML hasn't arrived yet.

Reach

ID	Title	Tier	Credit	Was
P2 `reach.canonical`	Canonical points to this page	Essential	self-referencing `<link rel="canonical">` → 1 · points elsewhere → 0.4 · missing → 0	`seo-canonical`

Read

ID	Title	Tier	Credit	Was
P3 `read.no-js-content`	Content is there without JavaScript	Essential	share of the draft's words in the server HTML: ≥ 95% → 1 · ≥ 80% → 0.5 · less → 0. **Not checked** without a draft (dashboard, fallback) or without the live page	new
P4 `read.landmarks`	Page structure is machine-readable	Recommended	landmarks present ÷ 4 (`<main>` , `<article>` , `<nav>` , `<header>`); unlabelled buttons/links are listed	`cq-agentic-readiness`
P7 `read.speed`	Page loads fast	Recommended	PageSpeed ≥ 70 → 1 · 50–69 → 0.6 · < 50 or not crawlable → 0. **Not checked** until PageSpeed runs, or when PSI errors	`cq-page-experience`

P5 and P6 (Markdown for agents) arrive in Phase 7.

P3 in detail (`compareBlocks` in `draft-diff.ts`). The draft includes content rendered by JavaScript and the server HTML doesn't (SPIKE-FINDINGS §3, item 4), so the gap is what crawlers that don't run

JavaScript miss. Both sides are `ParsedHtml.bodyBlocks`: every text block of `<body>`, header, nav, footer and hidden elements included, each with its nearest `data-component` (field and layout wrappers such as `helpers/fieldwrappers/richtextwrapper` are skipped, so a card's summary belongs to `relatedarticles`), capped at 50,000 chars.

- Text is compared lower-cased, with curly quotes and dashes normalised. A `{{…}}` token matches any one word. A draft block found anywhere in the joined live text is server-rendered, so blocks split differently on each side still match.
- A draft block of **4+ words** that isn't found is either an **edit** (word overlap ≥ 0.5 with a live block missing from the draft: counts as server content) or **client-only**. Shorter blocks (dates, "Read more", labels) aren't judged. Empty-state messages aren't counted and are listed as "Couldn't render in the editor".
- Text on the live page but not in the draft (the St. Patrick's Day paragraph) is never reported by P3.
- Flagged items group the client-only text by component.
- **On BrandX** (captures of 2026-11-01): the Banking article scores half credit, "44 of 522 words need JavaScript (91% in the server HTML)", flagged as `relatedarticles: 44 words` (the cards' titles and summaries). The published page's Related Articles section has only its heading and a loading spinner. St. Patrick's Day passes: its article listing is an empty field in the editor and adds no text.
- **Limitation:** text alone can't tell client-rendered content from new text that isn't published yet, so a brand-new unpublished component shows up as client-only until it's published. The result says so.

Understand

ID	Title	Tier	Credit	Was
P8 `understand.headings`	One clear H1 and logical headings	Essential	half for exactly one H1 (1/6 for several), half for no skipped levels	`seo-headings`
P9 `understand.title-meta`	Title and description are set	Essential	average of title (50–60 chars → 1 · 30–70 → 0.6 · other → 0.25 · missing → 0) and description (120–160 → 1 · other → 0.5 · missing → 0). Reads the Sitecore field first, the rendered tag as fallback	`seo-meta-title` + `seo-meta-description`
P10 `understand.json-ld`	Structured data is present and valid	Essential	mean over the schemas this page needs: valid → 1 · present but missing required properties → 0.5 · missing → 0. A block that doesn't parse caps it at 0.5 (fail). N/A when nothing is expected	`aeo-structured-data` + `aeo-faq-schema`
P11 `understand.metadata`	Page identity metadata is complete	Recommended	present ÷ 5 of `html lang`, `og:type`, `og:title`, `og:description`, `og:image`	`seo-og-tags`
P12 `understand.faq-quality`	FAQ answers are quotable	Recommended	answers of 40–300 chars ÷ all answers. N/A without Q&A pairs	`aeo-faq-answer-quality`
P13 `understand.question-headings`	Sections answer real questions	Recommended	≥ 3 question H2s or ≥ 50% of H2s → 1 · at least one → 0.5 · none → 0. N/A without H2s	`aeo-question-headings` (was off)

ID	Title	Tier	Credit	Was
P14 `understand.readability`	Easy to read and quote	Recommended	half: share of 20+-word paragraphs at ARI grade ≤ 13; half: average paragraph 40–120 words → 1 · 20–200 → 0.5. N/A without paragraphs	`cq-reading-grade` + the matrix's paragraph length
P15 `understand.image-alt`	Images are described	Recommended	images with alt ÷ all images. N/A without images	`seo-image-alt`

P10 in detail (fixes D2). The three old schema checks always passed and only printed a snippet. P10 reads the real `application/ld+json` blocks through the shared parser and expects:

- **BreadcrumbList** on every page except the home page. Required: `itemListElement` with `position` and `name` on each item.
- **FAQPage** when the page has two or more Q&A pairs. Required: `mainEntity` with `name` and `acceptedAnswer.text` on each question.
- **Article** (or any subtype, e.g. `BlogPosting`, `NewsArticle`) on article templates (template name "Article Detail Page"). Required: `headline` (or `name`).

The fix snippet contains only the missing or incomplete schemas (`buildBreadcrumbJsonLd`, `buildFaqPageJsonLd`, `buildArticleJsonLd` in `helpers.ts`). The site's Organization schema is S7, not part of P10.

Q&A pairs (P10, P12). `extractQaPairs` (`src/lib/html/extract-html.ts`) finds them in **any component**: an accordion toggle (`[aria-controls][aria-expanded]`) whose controlled panel holds the answer (collapsed, `aria-hidden` panels included), or a `<details>` whose `<summary>` is the question. Only toggles that read as a question (end in "?" or start with a question word) count, so menus and "Expand all" are skipped. On BrandX's FAQ page it finds all 8 questions. The AccordionItem datasource fetch in `usePageFields` is no longer used for scoring.

Trust

ID	Title	Tier	Credit	Was
P16 `trust.uniqueness`	Content is original, not generic	Recommended	AI rating `unique` → 1 · `borderline` → 0.5 · `commodity` → 0. Not checked until the AI has run	`cq-uniqueness`
P17 `trust.author`	Author is named and linked	Recommended	named and linked → 1 · named only → 0.5 · none → 0. **Article templates only** (N/A elsewhere)	`cq-author` (was off)
P18 `trust.freshness`	Content is recently updated	Recommended	≤ 180 days → 1 · ≤ 365 → 0.6 · ≤ 730 → 0.25 · older or no date → 0	`cq-freshness` (was off) + the matrix's bands
P19 `trust.depth`	Enough depth to be cited	Recommended	words in body paragraphs: ≥ 800 → 1 · ≥ 400 → 2/3 · ≥ 150 → 1/3 · fewer → 0	the matrix's content depth
P20 `trust.anchor-text`	Links say where they go	Recommended	generic anchors ≤ 20% → 1 · ≤ 40% → 0.5 · more → 0. N/A without links	`seo-anchor-text` (was info only)
P21 `trust.follow-up-gaps`	Answers the next question too	**Bonus**	follow-up questions covered ÷ all. Not checked until the AI has run	`cq-followup-gaps` (was off)

- **P16 body and caching.** The body is `ParsedHtml.mainText`: the text of `<main>` (or `<body>` without header, nav and footer), one block per line, entities decoded, capped at 20,000 chars. Hidden elements are skipped, except panels that a toggle controls (`aria-controls`), so collapsed FAQ answers are included (open item 3, fixed in Phase 4). In the panel it's the sanitised draft's main text, or live's when there's no draft; on the dashboard it's live's. `/api/ai-check` sends the first 3,000 chars to Claude on `ANTHROPIC_MODEL_BULK` (effort `low`, structured output). The panel calls `ai-check` once the body is ready and skips it when there's no text (console: `[scan] ai-check body: …`); see "Draft and live" for when it re-runs. Results are keyed by SHA-256 of the normalised title and text (`src/lib/ai/content-key.ts`) and cached in localStorage (`aiDiscovery:ai-results:<AI_CHECK_VERSION>`, shared by the panel and the dashboard) and in memory on the server, so an unchanged page doesn't call Claude again (console: `[ai-cache] hit …`). The AI re-run button skips both caches. A refusal returns `422` and the check stays not checked.

- **P17 author signals**, in order: the `author` of an Article-type JSON-LD node (a string, an inline Person/Organization, or an `@id` reference; linked when it has `url` or `sameAs`), then `<meta name="author">`, then a `rel="author"` link (which counts as the profile link).
- **P18 date sources**, in order: `<meta property="article:modified_time">`, a JSON-LD `dateModified`, `<meta name="last-modified">`, then Sitecore `__Updated`. `parseDate` (`helpers.ts`) reads ISO dates and Sitecore's compact `yyyyMMddTHHmmssZ` format.
- **P20 generic anchors.** `isGenericAnchor` (`helpers.ts`) matches the `ANCHOR_BLOCKLIST` exactly, plus a prefix rule limited to phrases that stay generic whatever follows ("click here …", "read more …", "see more …"). Descriptive anchors that start with a blocklisted word ("Download the 2026 report", "Register for the webinar") pass (SPIKE-FINDINGS §2).

`aeo-howto-schema` stays off (plan section 5.3); revisit only if HowTo content templates exist.

Site checks

Run once per site by `runSiteChecks` (`engine.ts`), with their own site score from the same pool maths (not shown since Phase 5). The dashboard lists them on the **Agent access** tab, and the ones that need work lead its Fix first list.

ID	Title	Tier	Credit	Was
S2 `reach.robots-ai-rules`	robots.txt lets AI search in	Essential	0 of 6 named AI crawlers blocked → 1 · 1–2 → 0.5 · 3+ → 0	the matrix's crawler permissions, moved as is; Phase 6 replaces it with bot groups by purpose
S7 `understand.org-schema`	Your organisation is described for AI	Recommended	see below	`aeo-organization-schema`

S7 `understand.org-schema` (`site/org-schema.ts`)

Graded on the **home page's live HTML** (the `getPageHTML` draft has no `<head>` scripts). The four rules agreed on 2026-10-31:

What the home page has	Result
Complete Organization schema	credit 1, pass
Organization schema with missing fields	partial credit, warning listing the fields, corrected snippet
No Organization schema (page fetched)	credit 0, **warning** ("Not added yet"), ready-to-paste snippet
Home page couldn't be fetched	**N/A** ("Couldn't check"), snippet

- **Credit:** 70% for the required fields (`name` ; `url` on the site's own domain, `www.` and subdomains allowed; `logo` as a URL or ImageObject) and 30% for the recommended ones (`sameAs` , `description` , `contactPoint`). Bonus fields (`address` as a PostalAddress, `legalName`) are listed as tips and not scored.
- **Detection:** every `application/ld+json` block, including `@graph` bundles, `@type` arrays (`["Organization", "LocalBusiness"]`), all 167 Organization subtypes from schema.org V30.1 (`schema-org-types.ts` , generated by `scripts/gen-schema-org-types.mjs`), and a `WebSite.publisher` that points to an Organization inline or by `@id` . The same entity declared twice under one `@id` counts once.
- **A block that doesn't parse** is a fail (credit 0) with the parse error shown.
- **Config is the expected value:** when `NEXT_PUBLIC_ORG_NAME` is set and the schema names a different organisation, or there are several Organization entries with conflicting names or URLs, the result is a warning and the credit is capped at 0.9.
- **Fix snippet:** starts from the schema already on the site and fills only the gaps (name, url, logo, description, sameAs) from `NEXT_PUBLIC_ORG_*` (`getOrganizationInfo` , with the site's name and domain as fallbacks). A `url` off the site's domain is replaced; other values are kept. With no schema at all it's `buildOrganizationJsonLd` .
- **In the panel** S7 shows only when the home page is open, in a "Site check — not in this page's score" section. The "site issues affecting this page" strip comes in Phase 6.
- **Evidence:** the BrandX home page has no JSON-LD at all (checked 2026-10-31 and 2026-11-01). The old check gave it 5/5; S7 gives it 0 with a warning and a snippet.

Shared JSON-LD parser (`json-ld.ts`)

`extract-html.ts` keeps every block as `ParsedHtml.jsonLdScripts` , either `{ ok: true, value }` or `{ ok: false, error }` , so parse errors are reported instead of dropped. `collectNodes` turns the blocks into typed nodes: each top-level object, each array entry and each `@graph` member (nested objects stay inside their parent; read them with `resolve` , which also follows `{"@id": …}` references).

`@type` is always a list with `schema:` / `https://schema.org/` prefixes removed. `isOrganization` and `isArticle` match subtypes. `typesOnPage` feeds the citation forecast prompt.

The view model and the panel UI (Phase 5)

`view-model.ts` turns reports into what the screens show. It never changes a score: everything comes from the points the engine computed.

- **Rows:** `toCheckRow` adds `display` (`pass` / `improve` / `fix` / `na` / `not-checked`, from `state` and `status`), `gain` (`maxPoints - points`) and `gainPts` (rounded, for "+N pts"; 0 hides it), plus the check's `why`, `owner` and `fix`. `STATUS_TEXT`, `OWNER_TEXT` and `SOURCE_TEXT` hold the words.
- **Top fixes:** `rankedFixes` keeps Improve and Fix rows worth at least 1 point, sorted by gain, then Essential first, then author fixes first. `topFixes` takes three.
- **Verdict:** `pageVerdict` gives the label from `labelForPct` and one sentence. `fixesToReady` adds the biggest gains to the score until it reaches 80: "N fixes would make this page agent-ready". If even every fix wouldn't, it says how many checks need work; at 80+ it says how many could still improve.
- **Fix buttons:** `fixButton`: **Generate** for P9 and P11 (the generator's fields; other `generate` fixes wait for Phase 9), **Copy snippet** for a `copy-snippet` fix with a `codePreview`, **Send to developer** for developer-owned fixes (copies `developerNote`: title, page URL, why, what we found, fix, details, code), otherwise **Show how** (opens and scrolls to the row). Copying uses `src/lib/clipboard.ts`, which falls back to `execCommand` because the Marketplace iframe doesn't delegate `clipboard-write`.
- **Panel** (`src/app/pages-contextpanel-extension/page.tsx`, `src/components/panel/`): header with the Draft/Live chip (Draft when `getPageHTML` gave a draft) and Re-scan; Radix tabs Score / Agent View (placeholder until Phase 7) / AI Assist (`MetaGenerator`). The tabs are force-mounted, so the generator keeps its state. `ScoreTab` holds the banner, `VerdictCard`, `CategoryBars` (toggle buttons that filter the list), `TopFixes` and the check groups (`CheckRow`), with a Needs work / All switch. Needs work also lists not-checked AI and PageSpeed rows, which have a **Run** button. N/A rows are listed under "Doesn't apply to this page". On the home page a Site check group shows S7.
- **"▲ N since last scan"** (`useScanDelta`, `src/hooks/useScoreHistory.ts`): when a scan settles (the page, its draft and the AI and PageSpeed checks have loaded), the stored score for the page (`aiDiscovery:pageScore:v1:<pageId>|<language>`) becomes the baseline and the new score is stored. Draft edits afterwards show against that baseline. Demo mode uses a fixed baseline (`DEMO_PREVIOUS_PAGE_SCORE`).
- **Shared components** (`src/components/shared/`): `StatusIcon` (icon, colour and word; `@mdi/js` paths), `ScoreRing` (fills on first render), `VerdictCard`, `CategoryBars`, `TopFixes`, `CheckRow`, `FixButton`, `Segmented`, `EmptyState`, the chips (`OwnerBadge`, `GainChip`, `SourceChip`,

`DeltaChip`), `AppMark`. Blok's components are in `src/components/ui/` and its theme tokens in `globals.css` (Sitecore/blok @ 0a7997e, light theme only).

- **Accessibility:** every status has an icon and a word; buttons and tabs are keyboard-reachable with a visible focus ring; the tab panels are focusable; the panel works at 320 px (the source chip wraps under the title). Motion is off under `prefers-reduced-motion`.
- **Dashboard:** `summarisePageReport` → `PageScoreSummary` (adds `issues` for Fix first) and `groupPageIssues` / `siteIssueGroups` / `issueNote`. See PAGE-MATRIX-TECHNICAL.

Meta & social generator

`src/components/MetaGenerator.tsx` (the panel's AI Assist tab) → `POST /api/generate-meta` (`{ field, pageTitle, bodyText, currentValue }` → `{ value, maxChars, overLimit }`), then `xmc.pages.updateFields` on **Populate field**. The logic is in `src/lib/server/generate-meta.ts`.

- **Model:** `ANTHROPIC_MODEL` (default `claude-opus-5-5`), effort `low`, structured output `{ value }`.
- **Length limits** (title 60, description 155, OG title 70, OG description 200): structured outputs can't enforce a maximum length, so the route checks it. An overrun gets one retry that tells the model the count ("62 characters, over the 60-character limit"). If the retry is still too long, it returns the value with `overLimit: true`, and the UI shows the count in amber. Values are never cut off.
- **Prompt caching:** the brand voice (system prompt) and the page title and text (first user block) carry `cache_control`, and the field instruction comes after them. Generating the other fields, or regenerating within 5 minutes, reads the cached prefix. All four fields use one output schema, so the cache is shared. The server log shows `cache_read=N` on each `[anthropic] generate-meta: <field>` line.
- **Body:** `bodyText` is the same text the uniqueness check judges: the draft's `mainText`, so unpublished edits are included, or live's without a draft. **Generate** is disabled while the scan is still reading it. If the scan finishes without text, the panel says so and generation falls back to the title and current values. The prompt tells the model to write from the page content and not copy the current value.
- **Brand voice:** `getBrandVoice()` (`src/lib/brand/brand-voice.ts`, server only) reads the Markdown file named by `BRAND_VOICE_FILE` (relative to the project root, read once per server instance) and the name in `BRAND_NAME`. With neither set, or if the file can't be read, it uses a neutral default voice. `brand/brandx.md` is the BrandX demo voice, and `next.config.ts` (`outputFileTracingIncludes`) bundles `brand/` into the Vercel function.
- **Log:** `[generate-meta] <field>: N chars of body text, brand voice: <source>`.

Server-side fetching (SSRF protection)

Every server route that fetches a caller-supplied URL (`preview-fetch` , `robots-check` , `footer-check` , `citation-forecast` , and `psi-check` for its Google call) uses `safeFetch` from `src/lib/server/safe-fetch.ts` :

- `http` / `https` only; no credentials in the URL.
- Blocks private, loopback, link-local (incl. `169.254.169.254`), CGNAT, multicast, reserved and documentation ranges, IPv4 and IPv6, including IPv4-mapped and NAT64/6to4/Teredo forms. The check runs in the undici Agent's DNS lookup at connect time, so DNS rebinding can't bypass it. IP-literal hosts (including `http://2130706433/` -style encodings) are checked before the request.
- Follows at most 3 redirects (re-checked each hop) and drops the caller's headers, such as the editing token, on a cross-origin hop.
- One timeout across redirects and body; bodies capped at 5 MB by default.
- Blocked input → `SafeFetchError` with `isBadInput` , and the route answers **400** `{ error: "blocked_url" }` . Fetch failures stay **502**, or the route's own fallback.
- `SAFE_FETCH_ALLOW_HOSTS` (comma-separated) skips the address check for named hosts. Use it only for local development.
- Tests: `src/lib/server/safe-fetch.test.ts` .

Tests

File	Covers
`src/lib/checks/scoring.test.ts`	pools, partial credit, weights, N/A and not checked, rescaling, bonus cap, categories, the gate
`src/lib/checks/page/page-checks.test.ts`	P10 real detection (D2), P9, N/A and not-checked cases, P17, P18, P3 (Related Articles, edits, empty states), body vs head sources, source chips, the fixture page and draft
`src/lib/checks/draft-diff.test.ts`	block comparison (client-only, edits, tokens, split blocks) and the "Draft differs" rule
`src/hooks/usePageHtml.test.ts`	`readWithRetry` : the first read retried while the canvas loads, giving up, no retry with a draft, a newer read taking over
`src/lib/html/sanitise-draft.test.ts`	the sanitiser on the BrandX FAQ, Banking article and St. Patrick's Day captures (`src/lib/html/fixtures/`), plus help text, Fallback hints, `[No text in field]` , `scEmpty*` placeholders, empty states, tokens, editor attributes
`src/lib/checks/brandx-pages.test.ts`	the whole pipeline on real BrandX draft + live pairs: P3 on the Related Articles cards, the footer token, the banner, P15 and P20
`src/lib/checks/site/org-schema.test.ts`	S7: the four rules, `@graph` , `@type` arrays, subtypes, `WebSite.publisher` , parse errors, config mismatch, duplicates, BrandX
`src/lib/checks/json-ld.test.ts`	the shared parser
`src/lib/checks/view-model.test.ts`	status words, Top fixes ranking, "N fixes would make this page agent-ready", fix buttons, the developer note, page issues in the summary, Fix first grouping
`src/lib/llms-txt/build.test.ts`	the "When to use this site" section and the score filter
`src/lib/relative-time.test.ts`	"N min ago" / "N h ago" / "N days ago"
`src/lib/checks/parity.test.ts`	panel and dashboard read the same fields and give the same score on the same inputs; a draft changes the panel's; the 80 / 60 thresholds
`src/lib/html/extract-html.test.ts`	`mainText` (collapsed answers included), entity decoding, `bodyBlocks` , Q&A pairs (BrandX accordion markup), metadata, JSON-LD blocks

Key files

File	Responsibility
`src/lib/checks/registry.ts`	Every check, in display order
`src/lib/checks/page/{reach,read,understand,trust}.ts`	Page checks P2–P21
`src/lib/checks/site/reach.ts`, `site/org-schema.ts`	Site checks S2, S7
`src/lib/checks/scoring.ts`, `gate.ts`, `engine.ts`	Pool maths, the gate, `runPageChecks` / `runSiteChecks`
`src/lib/checks/context.ts`	`buildPageContext` (draft → `body`), the shared field names, article/home detection
`src/lib/checks/draft-diff.ts`	Draft vs live block comparison: P3 and the "Draft differs" banner
`src/hooks/usePageHtml.ts`	Reads the draft with `getPageHTML`, debounced on field updates, retried while the canvas loads, with the live fallback
`src/components/panel/`	Score tab, the draft banner, the gate banner, the scan note
`src/components/shared/`	Status icon, score ring, verdict card, category bars, Top fixes, check row, fix button, chips
`src/hooks/useScoreHistory.ts`	"Since last scan" (panel) and the last 8 site scores (dashboard)
`src/lib/checks/json-ld.ts`, `schema-org-types.ts`	JSON-LD parsing and the schema.org subtype lists
`src/lib/checks/view-model.ts`	Reports → rows, Top fixes, verdicts, fix buttons, developer notes, dashboard summaries and Fix first
`src/lib/rule-engine/helpers.ts`	ARI, word counts, anchor blocklist, dates, snippet builders
`src/lib/rule-engine/types.ts`	Input types (`ParsedHtml`, `AiResults`, `PageData`, …)

File	Responsibility
`src/lib/rule-engine/test-fixture.ts`	Demo data for `?demo=1`, including the panel's demo draft (`makeTestDraft`: one edited heading, client-rendered Related Articles cards). Demo mode never starts the Marketplace client and doesn't auto-scan, so it makes no SitecoreAI or paid calls on load (`useDemoMode.ts`, `useMarketplaceClient.ts`)
`src/lib/score-thresholds.ts`	Shared 80 / 60 thresholds
`src/lib/html/extract-html.ts`	Parses HTML into `ParsedHtml` (server for live, browser for the draft)
`src/lib/html/sanitise-draft.ts`	Removes Page Builder artifacts from the draft before parsing
`src/lib/brand/organization.ts`, `brand-voice.ts`	Organization identity and brand voice config